

Phyllo – Product Analyst Internship Assignment

Candidate: Mohamed Aqeeb

Role: Product Analyst Intern

Task 1 – What doesn't match?

I compared the API documentation with the three response files and found the following discrepancies:

Issue	What the documentation says	What the data shows	Impact
Missing-order response	GET /v1/orders/{id} should return 404 when an order does not exist.	order_ord_9999.json returns 200 with {"order": null}.	High — clients may treat a missing order as a successful request.
Money format	Monetary amounts are integers in the smallest unit of the currency.	ord_1006 contains decimal values such as 44.0, 3.63, 5.99, and 53.62.	High — clients cannot consistently rely on the documented money format.
Pagination	has_more indicates whether another page should be requested.	Page 1 says has_more: false, but page 2 contains ord_1005 and ord_1006.	High — clients can stop early and miss valid orders.
Status values	status can be pending, shipped, delivered, or cancelled.	ord_1003 has status refunded.	Medium — strict validation may reject or mishandle the order.
Customer email	customer.email is always present.	ord_1005 has email: null.	Medium — integrations expecting a string need to handle null values.

Issue	What the documentation says	What the data shows	Impact
Order total	total should equal subtotal + tax + shipping.	For ord_1004, $6200 + 511 + 599 = 7310$, but total is 6810.	High — using the returned total can produce incorrect financial results.

Most serious issue

I would prioritise the **pagination issue**. A client following the documented `has_more` field would stop after page 1, even though additional orders exist on page 2. This can silently produce incomplete data and incorrect reports.

Task 2 – What's the total revenue?

Total Revenue: \$230.70

I used both pages because page 2 contains additional orders even though page 1 reports `has_more: false`.

I excluded `ord_1003` because it is marked as refunded.

For `ord_1004`, I did not use the reported total because it does not match the documented calculation:

$$6200 + 511 + 599 = 7310 \text{ cents} = \$73.10$$

For `ord_1006`, the response uses decimal values instead of the documented integer-smallest-unit format. The values reconcile to \$53.62, so I treated them as dollar values for this calculation. I would confirm this assumption with the API owner.

Revenue included

- `ord_1001` — \$54.70
- `ord_1002` — \$23.81
- `ord_1004` — \$73.10
- `ord_1005` — \$25.47
- `ord_1006` — \$53.62

Total: \$230.70

Task 3A – Reply to Priya

Subject: Re: Revenue reconciliation

Hi Priya,

I went through the order data and found a couple of issues that explain the reconciliation difference.

First, the API reports `has_more: false` on the first page even though another page of orders exists. A client following the documented pagination logic could therefore miss some orders.

Second, `ord_1004` reports a total of \$68.10, but its subtotal, tax and shipping add up to \$73.10. I used the component values for this order instead.

I also excluded `ord_1003` because it is marked as refunded.

Using both pages and these assumptions, I calculate **\$230.70** in revenue.

Best,
Mohamed Aqeeb

Task 3B – Bug Report

Incorrect `has_more` value on the orders endpoint

Endpoint: GET `/v1/orders`

Description:

The first response reports `has_more: false`, but additional orders are available on the next page.

Steps to reproduce:

1. Send GET `/v1/orders`.
2. Check the pagination fields in the response.
3. The response contains `has_more: false` and a `next_cursor`.
4. The next-page response contains `ord_1005` and `ord_1006`.

Observed result:

`has_more` is false even though another page of orders exists.

Expected result:

`has_more` should be true when another page contains orders and false only when there are no more orders.

Impact:

A client following the documented pagination logic will stop after the first page and miss valid orders. This can result in incomplete data and incorrect revenue reports.

Suggested fix:

Correct the pagination logic that generates `has_more` and add a test case covering a response where another page of orders exists.